

Ex No: 1

IMPLEMENT EIGHT QUEENS PROBLEM

Aim:

To Implement Eight Queens Problem using backtracking

Procedure:

1. Start.
2. Initialize an $n \times n$ board ($n = 8$) with all cells set to 0 (no queen placed).
3. Define `isSafe(mat, row, col)` to check whether a queen can be placed at `(row, col)`:
 - a. Scan all rows above the current row in the same column — if any cell is 1, return False.
 - b. Scan diagonally upward-left from `(row-1, col-1)` — if any cell is 1, return False.
 - c. Scan diagonally upward-right from `(row-1, col+1)` — if any cell is 1, return False.
 - d. If none of the above found a conflict, return True.
4. Define `placeQueens(row, mat)` using backtracking:
 - a. If `row == n`, all queens are placed successfully — return True.
 - b. Otherwise, for each column `i` from 0 to $n-1$ in the current row:
 - If `isSafe(mat, row, i)` is True, place a queen: `mat[row][i] = 1`.
 - Recursively call `placeQueens(row + 1, mat)`.
 - If the recursive call returns True, return True (solution found).
 - Otherwise, remove the queen (`mat[row][i] = 0`) and try the next column (backtrack).
 - c. If no column works for this row, return False.
5. Call `placeQueens(0, mat)` starting from row 0 to fill the board.
6. Print the final board, row by row, showing the placement of all 8 queens.
7. Stop.

Program:

N = 8 # Size of the chessboard (8x8)

Function to print the solution

```
def printSolution(board):
```

```
    for row in board:
```

```
        for i in range(N):
```

```
            print("Q" if row[i] else ".", end=" ") print()
```

```
        print() # Add a newline for readability
```

Function to check if a queen can be placed on board[row][col]

```
def isSafe(board, row, col):
```

```
    # Check the column
```

```
    for i in range(row):
```

```
        if board[i][col]:
```

```
            return False
```

```
    # Check the upper left diagonal
```

```
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
```

```
        if board[i][j]:
```

```
            return False
```

```
    # Check the upper right diagonal
```

```
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, N)):
```

```
        if board[i][j]:
```

```
            return False
```

```
    return True
```

```
# Function to solve the 8 Queens problem using backtracking
def solve(board, row):
    if row == N:
        printSolution(board)
        return True # Stop after finding one solution

    for col in range(N):
        if isSafe(board, row, col):
            board[row][col] = 1 # Place queen

            if solve(board, row + 1): # Recur to place next queen
                return True # Stop after first valid solution

            board[row][col] = 0 # Backtrack (remove queen)

    return False

# Main function to initialize the board and start solving the problem
def eightQueens():
    board = [[0 for _ in range(N)] for _ in range(N)]

    if solve(board, 0):
        print("One possible solution is:")
    else:
        print("Solution does not exist")

    # Calling the function
    eightQueens()
```

Output

```
1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
```

```
=== Code Execution Successful ===
```